

AI ASSISTED CODING

Assignment-20.5

HTNO:2303A510B4

Batch – 14

Lab 20 – Security Testing: Identifying Vulnerabilities in AI-Generated Code

Task 1 – Password Storage Vulnerability Check

PROMPT:

Generate a Python user registration script that stores username and password in a file or database without using hashing.

CODE:

```
16.py > ...
1  #Generate a Python user registration script that stores usern
2  def register_user():
3      username = input("Enter a username: ")
4      password = input("Enter a password: ")
5
6      with open("users.txt", "a") as file:
7          file.write(f"{username}:{password}\n")
8
9      print("User registered successfully!")
10 if __name__ == "__main__":
11     register_user()
12
```

PROBLEMS 10 OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\anith\OneDrive\Dokumen\ai-ac> & C:/Users/anith/.local/bin/py
neDrive/Dokumen/ai-ac/16.py
Enter a username: Anitha
Enter a password: 1234
User registered successfully!
```

Task 2 – Hardcoded Secrets Detection

PROMPT:

Generate a Python script that connects to an external API using a hardcoded API key or credentials in the code.

CODE:

```
import requests
api_key = "YOUR_HARDCODED_API_KEY_HERE"

# Using JSONPlaceholder as a public API for testing.
# It does not require an API key for basic GET requests.
api_url = "https://jsonplaceholder.typicode.com/posts"

# Define headers, including the API key (common for many APIs)
headers = {
    "Authorization": f"Bearer {api_key}", # Common for APIs using Bearer tokens
    "Content-Type": "application/json"
}

# Define any parameters or data for the request
params = {
    "userId": "1" # Example: Fetch posts by user ID 1
}

try:
    # Make the GET request
    response = requests.get(api_url, headers=headers, params=params)

    # Check if the request was successful (status code 200)
    response.raise_for_status()

    # Print the response content (e.g., JSON)
    print("API Response:")
    print(response.json())

except requests.exceptions.HTTPError as http_err:
    print(f"HTTP error occurred: {http_err}")
except requests.exceptions.ConnectionError as conn_err:
    print(f"Connection error occurred: {conn_err}")
except requests.exceptions.Timeout as timeout_err:
    print(f"Timeout error occurred: {timeout_err}")
except requests.exceptions.RequestException as req_err:
    print(f"An error occurred: {req_err}")

*** API Response:
[{'userId': 1, 'id': 1, 'title': 'sunt aut facere repellat provident occaecati excepturi optio reprehenderit', 'body': 'quia et suscipit\nsuscepit ut laborum sint dolor. Duis id labor et non conec...']
```

Task 3 – File Upload Security Validation

PROMPT:

Generate a simple Flask application that allows users to upload files without any validation or restrictions.

CODE:

```

import os
from flask import Flask, request, redirect, url_for, render_template

app = Flask(__name__)

# Define the upload folder. Create it if it doesn't exist.
UPLOAD_FOLDER = 'uploads'
if not os.path.exists(UPLOAD_FOLDER):
    os.makedirs(UPLOAD_FOLDER)
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER

@app.route('/')
def upload_form():
    return '''
    <!doctype html>
    <title>Upload New File</title>
    <h1>Upload New File</h1>
    <form method=post action="/upload" enctype=multipart/form-data>
        <input type=file name=file>
        <input type=submit value=Upload>
    </form>
    '''

@app.route('/upload', methods=['POST'])
def upload_file():
    if 'file' not in request.files:
        return redirect(request.url)
    file = request.files['file']
    if file.filename == '':
        return redirect(request.url)
    if file:
        filename = file.filename # No validation or sanitization!
        filepath = os.path.join(app.config['UPLOAD_FOLDER'], filename)
        file.save(filepath)
        return f'File {filename} uploaded successfully to {UPLOAD_FOLDER}!'
    return redirect(url_for('/'))

```

CODE:

To test this Flask application in Google Colab, you'll need to use a tool like `ngrok` to expose the local server running inside the Colab environment to the internet. Here's how you can do that:

1. Install `pyngrok` :

```
!pip install flask pyngrok
```

2. Run the Flask app with `ngrok` :

```
from pyngrok import ngrok
import threading
import time

# Kill any ngrok processes already running
ngrok.kill()

# Start ngrok tunnel
public_url = ngrok.connect(5000)
print(f" * ngrok tunnel available at: {public_url}")

# Function to run Flask app in a separate thread
def run_flask():
    app.run(port=5000)

# Start Flask app in a new thread
thread = threading.Thread(target=run_flask)
thread.daemon = True
thread.start()

# Give Flask a moment to start
time.sleep(3)
print(f" * Open the public URL to access your app: {public_url}")
```

After running the above cell, you will get a public URL. Click on that URL to access your Flask application and upload files. The files will be saved in the `uploads` folder in your Colab environment.

Task 4 – Command Injection Prevention

PROMPT:

Generate a Python script that takes user input and executes it as a system command using `os.system()` or `subprocess`.

CODE:

```

import subprocess

# Get user input for the command
user_command = input("Enter a system command to execute (e.g., 'mkdir my_directory'): ")

print(f"Executing command: {user_command}")

try:
    result = subprocess.run(user_command, shell=True, capture_output=True, text=True, check=True)

    print("\nCommand output:")
    print(result.stdout)
    if result.stderr:
        print("\nCommand error (stderr):")
        print(result.stderr)

except subprocess.CalledProcessError as e:
    print(f"\nError executing command: {e}")
    print(f"Stderr: {e.stderr}")
except FileNotFoundError:
    print(f"\nError: Command '{user_command.split()[0]}' not found.")
except Exception as e:
    print(f"\nAn unexpected error occurred: {e}")

```

```

Enter a system command to execute (e.g., 'mkdir my_directory'): mkdir --help
Executing command: mkdir --help

```

```

Command output:
Usage: mkdir [OPTION]... DIRECTORY...
Create the DIRECTORY(ies), if they do not already exist.

```

```

Mandatory arguments to long options are mandatory for short options too.
-m, --mode=MODE      set file mode (as in chmod), not a=rwx - umask
-p, --parents         no error if existing, make parent directories as needed
-v, --verbose         print a message for each created directory
-Z                   set SELinux security context of each created directory
                     to the default type
--context[=CTX]      like -Z, or if CTX is specified then set the SELinux
                     or SMACK security context to CTX
--help               display this help and exit
--version            output version information and exit

```

```

GNU coreutils online help: <https://www.gnu.org/software/coreutils/>
Full documentation <https://www.gnu.org/software/coreutils/mkdir>

```

Task 5 – Secure Input Handling in Web Forms

PROMPT:

Generate a basic HTML and Flask feedback form that accepts user input and displays it without validation or sanitization.

CODE:


```

from flask import Flask, request, render_template_string

app = Flask(__name__)

HTML_FORM = '''
<!doctype html>
<title>Feedback Form</title>
<h1>Submit Your Feedback</h1>
<form method=post action="/submit_feedback">
  <label for="name">Name:</label><br>
  <input type="text" id="name" name="name" size="50"><br><br>
  <label for="email">Email:</label><br>
  <input type="text" id="email" name="email" size="50"><br><br>
  <label for="feedback_text">Your Feedback:</label><br>
  <textarea id="feedback_text" name="feedback_text" rows="5" cols="50"></textarea><br><br>
  <input type="submit" value="Submit Feedback">
</form>
'''

HTML_DISPLAY = '''
<!doctype html>
<title>Feedback Received</title>
<h1>Feedback Received</h1>
<p><strong>Name:</strong> {{ name }}</p>
<p><strong>Email:</strong> {{ email }}</p>
<p><strong>Feedback:</strong></p>
<pre>{{ feedback_text }}</pre>
<p><a href="/feedback">Submit more feedback</a></p>
'''

@app.route('/feedback')
def feedback_form():
    return render_template_string(HTML_FORM)

@app.route('/submit_feedback', methods=['POST'])
def submit_feedback():
    user_name = request.form.get('name', 'N/A')
    user_email = request.form.get('email', 'N/A')
    user_feedback = request.form.get('feedback_text', 'No feedback provided')

    # WARNING: Displaying input without sanitization! This is for demonstration only.
    return render_template_string(HTML_DISPLAY, name=user_name, email=user_email, feedback_text=user_feedback)

```

```
# This will run the Flask application directly. Colab will provide a link to access it.
# Look for a link like 'Running on http://127.0.0.1:5000' or similar,
# and Colab will often generate an external URL you can click.
app.run(port=5000)
```

```
* Serving Flask app '__main__'
* Debug mode: off
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
INFO:werkzeug:Press CTRL+C to quit
ERROR:root:Unexpected exception finding object shape
Traceback (most recent call last):
  File "/usr/local/lib/python3.12/dist-packages/google/colab/_debugpy_repr.py", line 54, in get_shape
    shape = getattr(obj, 'shape', None)
            ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/werkzeug/local.py", line 318, in __get__
    obj = instance._get_current_object()
            ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/werkzeug/local.py", line 519, in _get_current_object
    raise RuntimeError(unbound_message) from None
RuntimeError: Working outside of request context.
```

This typically means that you attempted to use functionality that needed an active HTTP request. Consult the documentation on testing for information about how to avoid this problem.

```
ERROR:root:Unexpected exception finding object shape
Traceback (most recent call last):
  File "/usr/local/lib/python3.12/dist-packages/google/colab/_debugpy_repr.py", line 54, in get_shape
    shape = getattr(obj, 'shape', None)
            ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/werkzeug/local.py", line 318, in __get__
    obj = instance._get_current_object()
            ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/werkzeug/local.py", line 519, in _get_current_object
    raise RuntimeError(unbound_message) from None
RuntimeError: Working outside of request context.
```

This typically means that you attempted to use functionality that needed an active HTTP request. Consult the documentation on testing for information about how to avoid this problem.